

Lab 1: A 5-Detector on MNIST

Binary SGD, cross-validation, and the precision–recall trade-off.

Work in a **Jupyter** notebook. Number the exercises. Do notes **1.1** through **1.4** first. You will need `scikit-learn`, `numpy`, and `matplotlib`.

The original assignment PDF is [lab-1-assignment.pdf](#). A sample helper notebook (`SampleCode_Lab1`) was used in class for loading MNIST — `sklearn.datasets.fetch_openml("mnist_784")` is enough.

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.linear_model import SGDClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    roc_curve,
    roc_auc_score,
)
```

1. MNIST and a 5-detector (1 point)

Load MNIST (70,000 images, 784 pixels). Build `X` and `y`. Print their shapes. Plot a few digits.

Make a **binary** label: 1 if the digit is `5`, else 0. That classifier is the rest of the lab.

2. Train SGD (2 points)

First 60,000 rows are train; last 10,000 are test. Shuffle the **training** indices (and the matching labels).

Fit `SGDClassifier` on the 5 / not-5 task. Print the model.

3. Evaluate (2 points)

On the training labels (with cross-validation) and/or the test set, do **all** of these:

1. Accuracy with `cross_val_score`. Then implement the same idea yourself with `StratifiedKFold` and check that the numbers are close.
2. A confusion matrix.
3. Precision, recall, F1. Write one sentence about the precision–recall trade-off (raising the threshold does what?).
4. An ROC curve and an AUC.
5. The same ROC for `RandomForestClassifier`. Which curve is better on this 5-detector, and by how much?

A classifier that always says “not 5” is about 90% accurate. Do not quote accuracy alone.